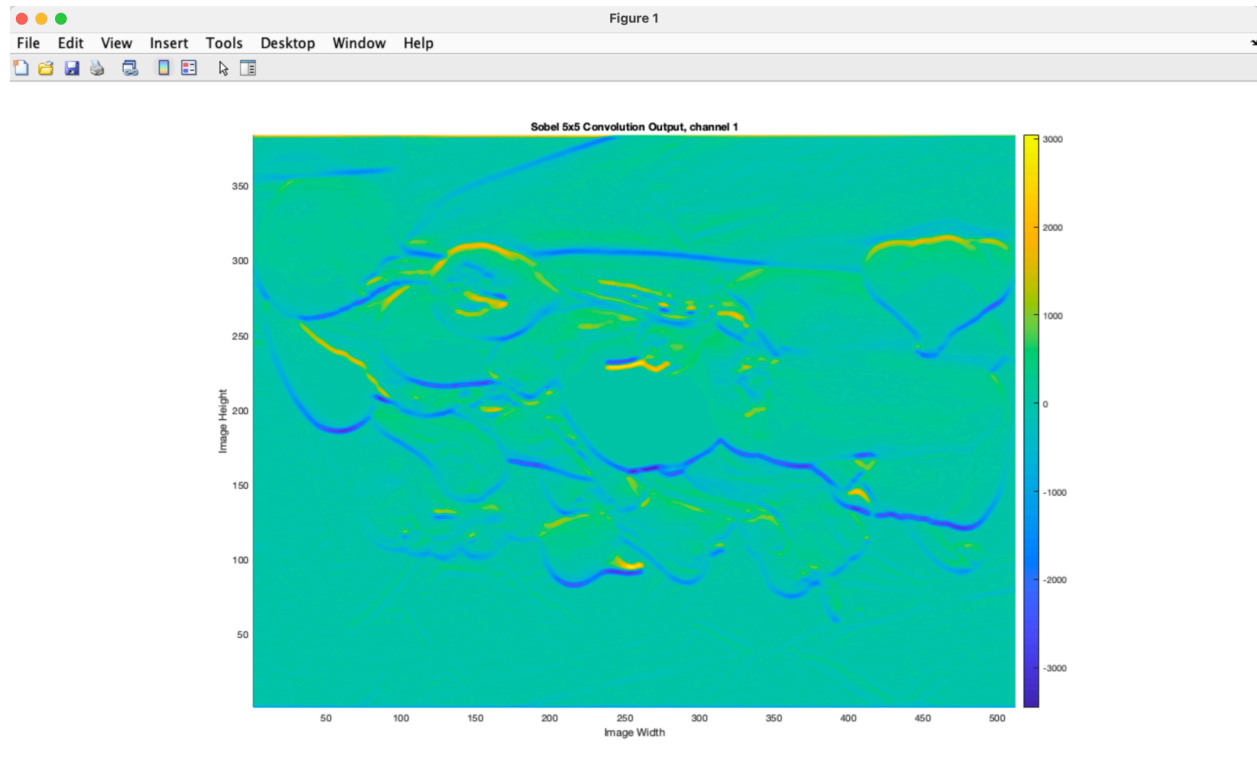


Brandon Nguyen, 827813045
COMPE596 - Spring 2026
3/29/2026

COMPE596 P07 - NVIDIA CUDA GPU Accelerated 2D Discrete Convolution

1) Matlab Plot



2) Code Listing

// Main Program

```
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include <cuda.h>
```

```
#define TILE_WIDTH 16
#define MASK_WIDTH 5
#define MASK_RADIUS 2
```

```
// CUDA Kernel for 2D convolution
```

```

__global__ void convolution(
    int *input,
    int *output,
    int *mask,
    int channels,
    int width,
    int height)
{
    int tx = blockIdx.x * blockDim.x + threadIdx.x;
    int ty = blockIdx.y * blockDim.y + threadIdx.y;

    if (tx >= width || ty >= height)
        return;

    for (int c = 0; c < channels; c++)
    {
        int sum = 0;

        for (int i = -MASK_RADIUS; i <= MASK_RADIUS; i++)
        {
            for (int j = -MASK_RADIUS; j <= MASK_RADIUS; j++)
            {
                int x = tx + j;
                int y = ty + i;

                if (x >= 0 && x < width && y >= 0 && y < height)
                {
                    int imageldx = (y * width + x) * channels + c;
                    int maskldx = (i + MASK_RADIUS) * MASK_WIDTH + (j +
MASK_RADIUS);

                    sum += input[imageldx] * mask[maskldx];
                }
            }
        }

        int outldx = (ty * width + tx) * channels + c;
        output[outldx] = sum;
    }
}

```

```

int main()
{
    int imageWidth = 512;
    int imageHeight = 384;
    int imageChannels = 3;

    int inputLength = imageWidth * imageHeight * imageChannels;

    // Allocate host memory
    int *hostInputImage = (int*)malloc(inputLength * sizeof(int));
    int *hostOutputImage = (int*)malloc(inputLength * sizeof(int));

    // Read input file
    FILE *f = fopen("peppers.dat", "r");
    if (!f)
    {
        printf("Error opening input file\n");
        return 1;
    }

    for (int i = 0; i < inputLength; i++)
    {
        fscanf(f, "%d", &hostInputImage[i]);
    }
    fclose(f);

    // Sobel 5x5 horizontal mask
    int hostMask[5][5] = {
        {2, 2, 4, 2, 2},
        {1, 1, 2, 1, 1},
        {0, 0, 0, 0, 0},
        {-1, -1, -2, -1, -1},
        {-2, -2, -4, -2, -2}
    };

    // Flatten mask
    int flatMask[25];
    for (int i = 0; i < 5; i++)
        for (int j = 0; j < 5; j++)

```

```

        flatMask[i * 5 + j] = hostMask[i][j];

// Allocate device memory
int *deviceInputImage, *deviceOutputImage, *deviceMask;

cudaMalloc((void**)&deviceInputImage, inputLength * sizeof(int));
cudaMalloc((void**)&deviceOutputImage, inputLength * sizeof(int));
cudaMalloc((void**)&deviceMask, 25 * sizeof(int));

// Copy to device
cudaMemcpy(deviceInputImage, hostInputImage,
           inputLength * sizeof(int), cudaMemcpyHostToDevice);

cudaMemcpy(deviceMask, flatMask,
           25 * sizeof(int), cudaMemcpyHostToDevice);

// Define grid/block
dim3 dimBlock(TILE_WIDTH, TILE_WIDTH);
dim3 dimGrid(
    (imageWidth + TILE_WIDTH - 1) / TILE_WIDTH,
    (imageHeight + TILE_WIDTH - 1) / TILE_WIDTH
);

// Launch kernel
convolution<<<dimGrid, dimBlock>>>(
    deviceInputImage,
    deviceOutputImage,
    deviceMask,
    imageChannels,
    imageWidth,
    imageHeight
);

cudaDeviceSynchronize();

// Copy result back
cudaMemcpy(hostOutputImage, deviceOutputImage,
           inputLength * sizeof(int), cudaMemcpyDeviceToHost);

// Write output file

```

```

FILE *out = fopen("peppers.out", "w");
for (int i = 0; i < inputLength; i++)
{
    fprintf(out, "%d\n", hostOutputImage[i]);
}
fclose(out);

// Cleanup
cudaFree(deviceInputImage);
cudaFree(deviceOutputImage);
cudaFree(deviceMask);

free(hostInputImage);
free(hostOutputImage);

printf("Convolution completed successfully.\n");

return 0;
}

```

// Makefile

```

NVCC = nvcc
TARGET = P07
SRC = P07.cu
CFLAGS = -O2

all: $(TARGET)

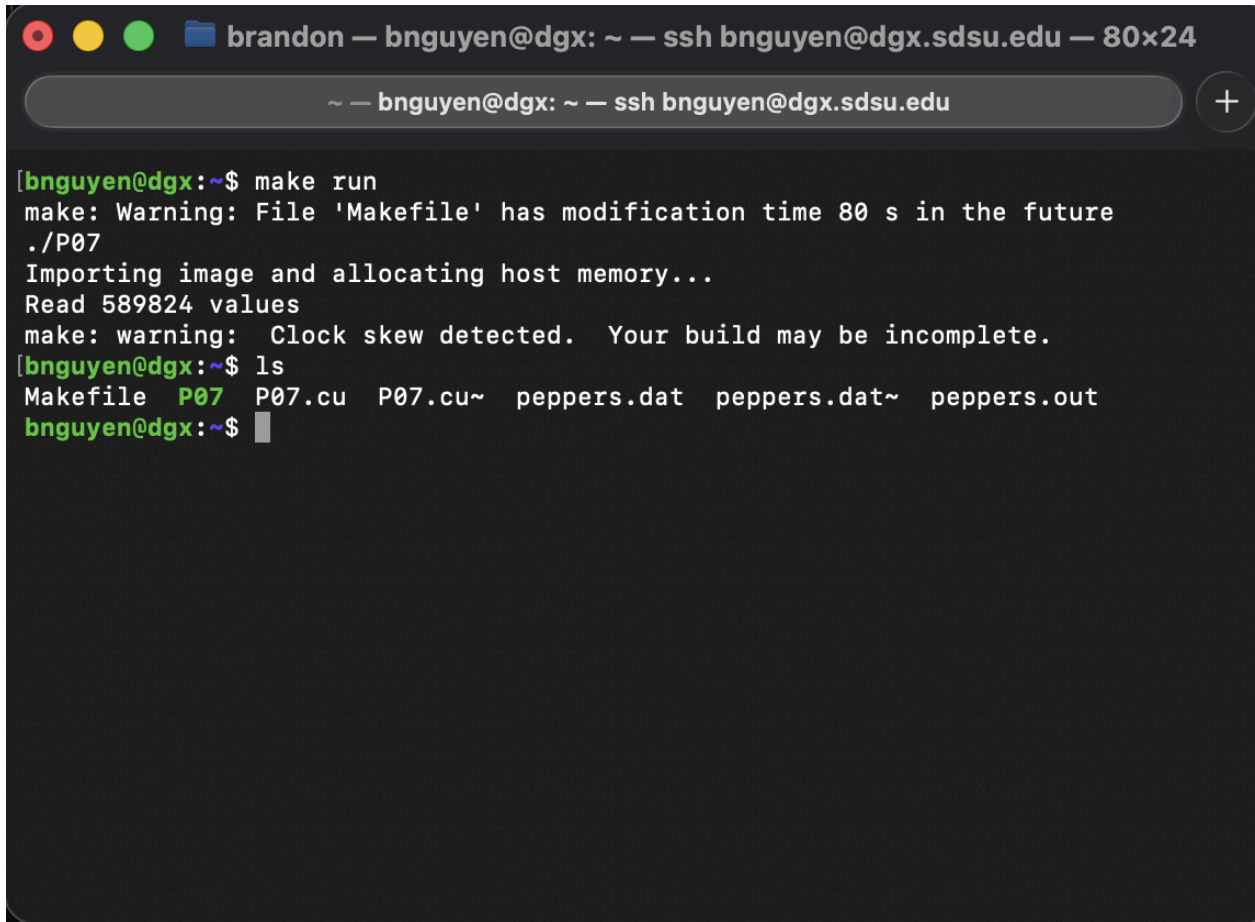
$(TARGET): $(SRC)
    $(NVCC) $(CFLAGS) -o $(TARGET) $(SRC)

run: $(TARGET)
    ./$(TARGET)

clean:
    rm -f $(TARGET) peppers.out

```

3) Program Output

A terminal window with a dark background and light-colored text. The window title bar shows 'brandon — bnguyen@dgx: ~ — ssh bnguyen@dgx.sdsu.edu — 80x24'. The terminal content shows the execution of 'make run', which includes warnings about a future modification time and clock skew, followed by the command 'ls' showing files like 'P07', 'peppers.dat', and 'peppers.out'.

```
brandon — bnguyen@dgx: ~ — ssh bnguyen@dgx.sdsu.edu — 80x24
~ — bnguyen@dgx: ~ — ssh bnguyen@dgx.sdsu.edu

bnguyen@dgx:~$ make run
make: Warning: File 'Makefile' has modification time 80 s in the future
./P07
Importing image and allocating host memory...
Read 589824 values
make: warning: Clock skew detected. Your build may be incomplete.
bnguyen@dgx:~$ ls
Makefile  P07  P07.cu  P07.cu~  peppers.dat  peppers.dat~  peppers.out
bnguyen@dgx:~$
```

4) MATLAB Plot Script

```
clc
close all
m = 384; % height
n = 512; % width
c = 3; % channels
P = zeros(m,n,c);
fid = fopen('peppers.out','r');
i = 1; j = 1; k = 1;
for l = 1:m*n*c,
    p = fscanf(fid,'%d', 1);
    P(i,j,k) = p;
    if mod(k,c) == 0
        k = 1;
        j = j + 1;
        if j == n + 1
```

```

        j = 1;
        i = i + 1;
        if i == m + 1
            break;
        end
    end
else
    k = k + 1;
end
end
fclose(fid);
[M,N] = meshgrid(1:1:n,1:1:m);
c = 1;
surf(M,N,P(:,c));
xlabel('Image Width')
ylabel('Image Height')
title(['Sobel 5x5 Convolution Output, channel ',num2str(c)])
grid on
grid minor
set(gcf,'color','w')
axis equal
colorbar
shading interp
rotate3d
view(0,90);
set(gcf,'position',[10,10,1280,768])
figure(1)

```

5) Discussion

In P07, we are tasked with observing GPU accelerated 2D discrete convolution using NVIDIA CUDA to convolve a PNG image with a Sobel 5x5 horizontal convolution kernel for edge detection. First, we begin by saving the given peppers image as a PNG file. Then we convert the peppers PNG file into a peppers DAT file using a MATLAB script, since the CUDA kernel looks for and takes a DAT type file only. We are then expected to write a CUDA kernel program that will take the DAT file and implement the 2D CUDA convolution, resulting in a peppers OUT file. We then take the newly generated peppers OUT file and plot the convolution output as a 3D surface using the provided MATLAB script. Taking everything into consideration, the seventh programming assignment furthered our understanding of CUDA kernel programming when it comes to working with PNG files and convolution.